

Architektura Komponentow - Home Budget

Szczegolowa dokumentacja funkcjonalnosci, API, walidacji i komponentow znajduje sie w pliku FEATURES.md

Diagram architektury komponentow

```
graph TB
  subgraph Router["Router (react-router)"]
    direction TB
    R[" / "] --> Layout
  end

  subgraph Layout["Layout"]
    direction TB
    L[Layout] --> |Outlet| Pages
  end

  subgraph Pages["Strony (Pages)"]
    direction TB
    Dashboard[Dashboard]
    TransactionList[TransactionList]
    TransactionAdd[TransactionAdd]
    TransactionDetail[TransactionDetail]
    CategoryList[CategoryList]
    CategoryAdd[CategoryAdd]
  end
```

```
subgraph SharedComponents["Komponenty wspoldzielone"]
    Button[Button]
    InputField[InputField]
    SummaryCard[SummaryCard]
end
```

```
subgraph TransactionComponents["Komponenty Transakcj"]
    TransactionForm[TransactionForm]
    TransactionItem[TransactionItem]
end
```

```
subgraph CategoryComponents["Komponenty Kategorii"]
    CategoryForm[CategoryForm]
    CategoryItem[CategoryItem]
end
```

```
subgraph FluxArchitecture["Flux Architecture"]
    Store[Store]
    Dispatcher[Dispatcher]
    Actions[Actions]
end
```

```
subgraph Services["Serwisy"]
    API[API - Axios]
end
```

```
Dashboard --> SummaryCard
TransactionList --> TransactionItem
TransactionList --> TransactionForm
TransactionAdd --> TransactionForm
CategoryList --> CategoryItem
CategoryList --> CategoryForm
CategoryAdd --> CategoryForm
```

```
TransactionForm --> InputField
TransactionForm --> Button
CategoryForm --> InputField
CategoryForm --> Button
```

TransactionItem --> Button

CategoryItem --> Button

Pages --> Actions

Actions --> Dispatcher

Dispatcher --> Store

Store --> Pages

Actions --> API

Hierarchia komponentow

graph TD

App[App] --> RouterProvider

RouterProvider --> Layout

Layout --> Dashboard

Layout --> TransactionList

Layout --> TransactionAdd

Layout --> TransactionDetail

Layout --> CategoryList

Layout --> CategoryAdd

Dashboard --> SummaryCard

TransactionList --> TransactionItem

TransactionList --> TransactionForm

TransactionAdd --> TransactionForm

CategoryList --> CategoryItem

CategoryList --> CategoryForm

CategoryAdd --> CategoryForm

```
TransactionForm --> InputField
TransactionForm --> Button
CategoryForm --> InputField
CategoryForm --> Button
```

Architektura Flux

```
sequenceDiagram
    participant Component as Komponent React
    participant Actions as Actions
    participant Dispatcher as Dispatcher
    participant Store as Store
    participant API as API (Axios)

    Component->>Actions: wywołanie akcji (np. fetchTrans
    Actions->>API: zadanie HTTP
    API-->>Actions: odpowiedz
    Actions->>Dispatcher: dispatch(action)
    Dispatcher->>Store: wykonanie reducera
    Store-->>Component: emitChange()
    Component->>Component: setState(store.getState())
```

Przepływ danych

```
flowchart LR
    subgraph Frontend
        UI[Komponenty UI]
        Store[Store]
```

```
        Actions[Actions]
    end

    subgraph Backend
        JSONServer[JSON Server]
        DB[(db.json)]
    end

    UI -->|"wywołuje"| Actions
    Actions -->|"HTTP request"| JSONServer
    JSONServer -->|"CRUD"| DB
    JSONServer -->|"HTTP response"| Actions
    Actions -->|"dispatch"| Store
    Store -->|"emitChange"| UI
```

Struktura katalogow

```
src/
├── components/                # Komponenty wielokrotnego uzytku
│   ├── Button.tsx
│   ├── InputField.tsx
│   ├── SummaryCard.tsx
│   ├── categories/
│   │   ├── CategoryForm.tsx
│   │   └── CategoryItem.tsx
│   └── transactions/
│       ├── TransactionForm.tsx
│       └── TransactionItem.tsx
├── flux/                      # Architektura Flux
│   ├── actions.ts
│   ├── Dispatcher.ts
│   └── Store.ts
├── layout/
│   └── Layout.tsx
```

```
├─ pages/
│   ├─ CategoryAdd.tsx
│   ├─ CategoryList.tsx
│   ├─ Dashboard.tsx
│   ├─ TransactionAdd.tsx
│   ├─ TransactionDetail.tsx
│   └─ TransactionList.tsx
├─ services/
│   └─ Api.ts
├─ App.css
├─ main.tsx
└─ router.tsx
```